

CO2 – AT3
MLA 0201 – Fundamentals of Machine Learning

Annexure A

Debugging & Optimisation Task

Q1. A real estate company developed a linear regression model to predict house prices, but the model is showing high prediction error.

- (i) Identify possible causes for the high error (e.g., multicollinearity, outliers, poor feature selection).
- (ii) Analyze the dataset/model behavior to detect these issues.
- (iii) Apply suitable optimization techniques (such as feature scaling, removing outliers, or adding relevant features) to improve the model performance. (5 Marks-8 Mins)

Q2. An email classification system using a linear model is incorrectly classifying several spam emails as non-spam.

- (i) Identify the reasons for misclassification (e.g., improper decision boundary, imbalanced data).
- (ii) Examine the current decision boundary and classification errors.
- (iii) Refine the model using optimization techniques such as adjusting thresholds, feature engineering, or re-training with balanced data. (5 Marks-8 Mins)

Q3. A Naïve Bayes classifier is giving poor accuracy due to missing or zero probability values in the dataset.

- (i) Identify the issue related to missing or zero probabilities.
- (ii) Analyze how these values affect classification results.
- (iii) Apply appropriate debugging and optimization methods such as Laplace smoothing to improve accuracy. (5 Marks-8 Mins)

Q4. A discriminant function is unable to clearly separate two classes, leading to incorrect classifications.

- (i) Identify possible reasons for poor class separation (e.g., overlapping features, incorrect weights).
- (ii) Analyze the decision boundary and feature contribution.

- (iii) Improve the discriminant function using optimization strategies such as adjusting weights, feature transformation, or normalization.(5 Marks-8 Mins)

Q5.A Bayesian logistic regression model developed for customer churn prediction is suffering from overfitting, resulting in poor generalization.

- (i) Identify signs and causes of overfitting in the model.
- (ii) Analyze model performance on training vs testing data.
- (iii) Apply debugging and regularization techniques (such as L1/L2 regularization or cross-validation) to improve model performance.(5 Marks-8 Mins)

Q6.A linear regression model built for predicting sales is performing poorly because the features (e.g., advertising budget in thousands and number of outlets in units) are on different scales.

- (i) Identify how feature scaling issues affect model performance.
- (ii) Analyze the impact of unscaled features on regression coefficients.
- (iii) Apply appropriate optimization techniques such as normalization or standardization to improve the model.(5 Marks-8 Mins)

Q7.A binary classification model for fraud detection is biased towards the majority class, leading to poor detection of fraudulent cases.

- (i) Identify the problem caused by class imbalance.
- (ii) Evaluate model performance using suitable metrics (precision, recall, F1-score).
- (iii) Apply optimization techniques such as resampling (oversampling/undersampling) or class weighting to improve performance.(5 Marks-8 Mins)

Q8.A Naïve Bayes classifier is underperforming because some input features are highly correlated, violating the independence assumption.

- (i) Identify how correlated features affect Naïve Bayes performance.
- (ii) Analyze feature relationships in the dataset.
- (iii) Apply optimization techniques such as feature selection or dimensionality reduction to improve accuracy. (5 Marks-8 Mins)

Q9.A logistic regression model is unable to clearly separate two classes due to overlapping feature distributions.

- (i) Identify the cause of overlapping decision boundaries.
- (ii) Analyze the feature space and classification errors.
- (iii) Improve the model using techniques such as feature transformation, polynomial features, or regularization. (5 Marks-8 Mins)

Q10. A discriminant analysis model is too simple and fails to capture the complexity of the dataset, resulting in low accuracy.

- (i) Identify the signs of underfitting in the model.
- (ii) Analyze model bias and feature limitations.
- (iii) Apply optimization strategies such as adding features, increasing model complexity, or using nonlinear transformations. (5 Marks-8 Mins)

1. Linear Regression – High House Price Prediction Error

(i) Problem Identification

High prediction error may be caused by:

- **Multicollinearity** – two or more features are highly correlated.
- **Outliers** – unusually expensive or cheap houses can distort the regression line.
- **Poor feature selection** – important features such as location, number of rooms, or house age may be missing.
- **Different feature scales** – features with very different ranges can affect optimization.
- **Non-linear relationships** – house price may not increase linearly with every feature.

(ii) Debugging and Analysis

The dataset can be analyzed using:

- Correlation matrix to detect highly correlated features.
- Box plots or IQR to identify outliers.
- Train-test split to compare training and testing errors.
- MAE, MSE and RMSE to measure prediction performance.
- Residual analysis to check whether prediction errors are randomly distributed.

```
from sklearn.metrics import mean_squared_error, r2_score

pred = model.predict(X_test)

print("RMSE:", mean_squared_error(y_test, pred) ** 0.5)

print("R2 Score:", r2_score(y_test, pred))

print("Correlation:\n", X.corr())
```

(iii) Optimization

The model can be improved by:

- Removing extreme outliers using IQR.

- Removing one of highly correlated features.
- Adding relevant features such as location, house age and number of rooms.
- Applying **StandardScaler** or **MinMaxScaler**.
- Using regularization such as Ridge or Lasso.

```
from sklearn.preprocessing import StandardScaler  
  
from sklearn.linear_model import Ridge  
  
scaler = StandardScaler()  
  
X_train = scaler.fit_transform(X_train)  
  
X_test = scaler.transform(X_test)  
  
model = Ridge(alpha=1.0)  
  
model.fit(X_train, y_train)
```

Result: Removing noise, selecting useful features and regularizing the model reduces prediction error and improves generalization.

2. Linear Model – Spam Email Misclassification

(i) Problem Identification

Spam emails may be classified as non-spam because of:

- Incorrect decision boundary.
- Insufficient or irrelevant features.
- Imbalanced spam/non-spam data.
- Similar vocabulary between spam and legitimate emails.
- Poorly selected classification threshold.

(ii) Debugging and Analysis

Analyze the **confusion matrix**:

- **True Positive (TP):** Spam correctly identified.
- **True Negative (TN):** Non-spam correctly identified.
- **False Positive (FP):** Normal email classified as spam.
- **False Negative (FN):** Spam classified as non-spam.

Since missing spam is the main problem, **recall for the spam class** is important.

```
from sklearn.metrics import confusion_matrix, classification_report

pred = model.predict(X_test)

print(confusion_matrix(y_test, pred))

print(classification_report(y_test, pred))
```

(iii) Optimization

- Adjust the classification threshold.
- Add features such as spam keywords, URL count, special-character count and email length.

- Balance the training dataset.
- Use class weights.
- Retrain the model using better features.

```
from sklearn.linear_model import LogisticRegression  
  
model = LogisticRegression(class_weight="balanced")  
  
model.fit(X_train, y_train)
```

Result: Better features and balanced training reduce false negatives and improve spam detection recall.

3. Naïve Bayes – Missing or Zero Probabilities

(i) Problem Identification

Naïve Bayes calculates:

If any conditional probability becomes **zero**, the entire product becomes zero.

Missing values can also produce incorrect probability calculations.

(ii) Debugging and Analysis

Check:

- Missing values in the dataset.
- Features having zero occurrences for a particular class.
- Probability tables generated by the classifier.
- Classification results before and after smoothing.

For example:

Then:

even if other words strongly indicate spam.

(iii) Optimization – Laplace Smoothing

Laplace smoothing prevents zero probabilities:

where **V** is the number of possible feature values.

```
from sklearn.naive_bayes import MultinomialNB

model = MultinomialNB(alpha=1.0)

model.fit(X_train, y_train)

pred = model.predict(X_test)
```

Here, $\alpha=1$ provides Laplace smoothing.

Result: Zero probabilities are avoided, making the classifier more robust and improving classification accuracy.

4. Discriminant Function – Poor Class Separation

(i) Problem Identification

Poor class separation can occur because:

- Features of the two classes overlap.
- Incorrect discriminant weights are used.
- Features have different scales.
- Irrelevant features introduce noise.
- The selected features do not sufficiently distinguish the classes.

A discriminant function can be represented as:

The class is selected based on the value of .

(ii) Debugging and Analysis

Analyze:

- Feature distributions of both classes.
- Decision boundary.
- Contribution of each feature through its weight.
- Misclassified samples.
- Correlation between features.

A large absolute weight indicates that the corresponding feature has a strong influence on the decision.

(iii) Optimization

- Normalize or standardize features.
- Adjust or retrain weights.
- Remove irrelevant features.
- Apply feature transformation.

- Add more discriminative features.

```
from sklearn.preprocessing import StandardScaler

from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)

model = LinearDiscriminantAnalysis()

model.fit(X_train, y_train)
```

Result: Better-scaled and more informative features produce a clearer decision boundary and improve classification accuracy.

5. Bayesian Logistic Regression – Customer Churn Overfitting

(i) Problem Identification

Signs of overfitting:

- Very high training accuracy.
- Much lower testing accuracy.
- Large difference between training and validation performance.
- Model performs poorly on unseen customers.

Causes include:

- Too many features.
- No regularization.
- Small training dataset.
- Noisy or irrelevant features.
- Excessive model complexity.

(ii) Debugging and Analysis

Compare:

```
train_score = model.score(X_train, y_train)

test_score = model.score(X_test, y_test)

print("Training:", train_score)

print("Testing:", test_score)
```

For example:

Training accuracy = 98%

Testing accuracy = 72%

This large difference indicates overfitting.

(iii) Optimization

Use:

- **L1 regularization** for feature selection.
- **L2 regularization** to reduce large coefficients.
- Cross-validation.
- Feature selection.
- More training data if available.

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(

    penalty="l2",

    C=0.1,

    max_iter=1000

)

model.fit(X_train, y_train)
```

Cross-validation can be used to select the best regularization strength.

Result: Regularization controls model complexity and improves performance on unseen customer data.

6. Linear Regression – Features on Different Scales

(i) Problem Identification

Suppose the features are:

- Advertising budget = ₹10,000–₹1,00,000
- Number of outlets = 1–100

These features have very different scales.

Large-scale features can dominate the optimization process and cause inefficient gradient-based learning.

(ii) Debugging and Analysis

Before scaling:

The coefficients may have very different numerical magnitudes because the input features use different units.

Check:

```
print(X_train.min())  
  
print(X_train.max())
```

If the ranges differ significantly, scaling is appropriate.

(iii) Optimization

```
from sklearn.preprocessing import StandardScaler  
  
from sklearn.linear_model import LinearRegression  
  
scaler = StandardScaler()  
  
X_train_scaled = scaler.fit_transform(X_train)  
  
X_test_scaled = scaler.transform(X_test)  
  
model = LinearRegression()  
  
model.fit(X_train_scaled, y_train)
```

Result: Scaling puts features on comparable ranges, improves numerical stability and makes optimization more efficient.

7. Fraud Detection – Class Imbalance

(i) Problem Identification

Fraud datasets usually contain many legitimate transactions and very few fraudulent transactions.

Example:

- Normal = 9,500
- Fraud = 500

A model predicting every transaction as normal can achieve **95% accuracy**, but it completely fails to detect fraud.

Therefore, accuracy alone is misleading.

(ii) Debugging and Analysis

Use:

Precision

Recall

F1-score

```
from sklearn.metrics import classification_report  
  
print(classification_report(y_test, pred))
```

For fraud detection, **recall is especially important** because missing fraudulent transactions is costly.

(iii) Optimization

Possible solutions:

- Oversampling the minority class.
- Undersampling the majority class.
- SMOTE.
- Class weighting.

```
from sklearn.linear_model import LogisticRegression  
  
model = LogisticRegression(class_weight="balanced")  
  
model.fit(X_train, y_train)
```

Result: The model gives greater importance to fraudulent transactions and improves fraud detection recall and F1-score.

8. Naïve Bayes – Highly Correlated Features

(i) Problem Identification

Naïve Bayes assumes that features are conditionally independent:

When features are highly correlated, this assumption is violated.

For example:

- annual_income
- monthly_income

contain related information.

Naïve Bayes may effectively count the same information multiple times.

(ii) Debugging and Analysis

Use a correlation matrix:

```
import pandas as pd  
  
correlation = X.corr()  
  
print(correlation)
```

Features with correlation close to **+1 or -1** may contain redundant information.

(iii) Optimization

Use:

- Feature selection.
- Remove highly correlated features.
- PCA for dimensionality reduction.
- Select the most informative features.


```
from sklearn.feature_selection import SelectKBest, chi2

selector = SelectKBest(chi2, k=10)

X_train_new = selector.fit_transform(X_train, y_train)

X_test_new = selector.transform(X_test)
```

Result: Removing redundant features reduces violation of the independence assumption and can improve Naïve Bayes performance.

9. Logistic Regression – Overlapping Classes

(i) Problem Identification

Logistic regression normally creates a linear decision boundary:

If the feature distributions of two classes overlap heavily, a straight line cannot separate them effectively.

(ii) Debugging and Analysis

Analyze:

- Feature distributions.
- Misclassified samples.
- Confusion matrix.
- Decision boundary.
- Relationship between features and target.

A high number of both false positives and false negatives indicates poor class separation.

(iii) Optimization

Use:

- Feature transformation.
- Polynomial features.
- Feature interaction.
- Regularization.
- Feature selection.

```
from sklearn.preprocessing import PolynomialFeatures  
  
from sklearn.linear_model import LogisticRegression  
  
poly = PolynomialFeatures(degree=2)  
  
X_train_poly = poly.fit_transform(X_train)  
  
X_test_poly = poly.transform(X_test)  
  
model = LogisticRegression(C=1.0, max_iter=1000)  
  
model.fit(X_train_poly, y_train)
```

Polynomial features allow logistic regression to model more complex boundaries.

Result: Feature transformation increases model flexibility and can improve classification when classes are not linearly separable.

10. Discriminant Analysis – Underfitting

(i) Problem Identification

Underfitting occurs when the model is **too simple** to represent the actual relationship in the data.

Signs include:

- Low training accuracy.
- Low testing accuracy.
- High bias.
- Important features are missing.
- Model cannot capture nonlinear relationships.

Example:

Training accuracy = 65%

Testing accuracy = 63%

Since both are low, the model is likely underfitting.

(ii) Debugging and Analysis

Compare training and testing performance:

```
print("Training Accuracy:",model.score(X_train, y_train))  
  
print("Testing Accuracy:",model.score(X_test, y_test))
```

If both scores are low, investigate:

- Number of features.
- Feature quality.
- Model complexity.
- Nonlinear relationships.
- Data representation.

(iii) Optimization

Improve the model by:

- Adding relevant features.
- Applying nonlinear transformations.
- Creating interaction features.
- Increasing model complexity.
- Using a more flexible discriminant model if necessary.

```
from sklearn.preprocessing import PolynomialFeatures  
  
poly = PolynomialFeatures(degree=2)  
  
X_train_new = poly.fit_transform(X_train)  
  
X_test_new = poly.transform(X_test)
```

Result: Adding informative features and increasing model flexibility reduces bias and improves both training and testing performance.